

Progetto APS: Sistema di E-Voting per Referendum Universitario

Simone Donato, Alessio Ferrara

Giugno 2026

Indice

1	WP1: Definizione del Modello di Sistema e Analisi delle Minacce	3
1.1	Scenario Applicativo	3
1.2	Attori del Sistema	3
1.3	Funzionalità del Sistema	3
1.3.1	Requisiti Funzionali	3
1.3.2	Requisiti Non Funzionali	4
1.4	Threat Models (Modello di Minaccia)	4
1.5	Proprietà del Sistema da Preservare Sotto Attacco	5
2	WP2: Progettazione della Soluzione	6
2.1	Strategia di Generazione e Distribuzione delle Chiavi	6
2.2	Identificazione dello Studente e Fase di Registrazione (Blind Signature con FDH)	6
2.3	Formato dei Messaggi e della Scheda di Voto	7
2.3.1	Struttura Dati (JSON)	7
2.3.2	Operazione di Cifratura (Schema Ibrido AEAD)	8
2.4	Flusso di Comunicazione e Registrazione del Voto	8
2.5	Meccanismo di Sostituzione o Annullamento	9
2.6	Procedura di Scrutinio e Resilienza Operativa	10
2.7	Meccanismo di Verifica Individuale e Universale (Bulletin Board Autenticato)	11
3	WP3: Analisi di Sicurezza e Mitigazione delle Minacce	13
3.1	Obiettivo dell'Analisi	13
3.2	Analisi delle Proprietà Crittografiche	13
3.2.1	Anonimato e Unlinkability	13
3.2.2	Integrità e CCA-Security	13
3.2.3	Anti-Coercizione e Receipt-Freeness	13
3.3	Risoluzione dei Threat Models (TM)	14
3.4	Analisi delle Alternative Progettuali e Trade-off	16
4	WP4: Implementazione e Prestazioni	18
4.1	Ambiente di Sviluppo e Tecnologie Utilizzate	18
4.1.1	Stack Tecnologico	18
4.2	Architettura del Prototipo	18

4.3	Flusso del Protocollo Implementato	19
4.4	Primitive Crittografiche Impiegate	20
4.5	Dimensione dei Messaggi Scambiati	20
4.6	Prestazioni Sperimentali	20
	4.6.1 Analisi di Scalabilità dello Scrutinio	21
4.7	Validazione Funzionale e di Sicurezza	21
4.8	Riepilogo	22

1 WP1: Definizione del Modello di Sistema e Analisi delle Minacce

1.1 Scenario Applicativo

Il presente pacchetto di lavoro definisce l'architettura logica e i confini di sicurezza per un sistema di voto elettronico (E-Voting) centralizzato ma crittograficamente verificabile, configurato per consultazioni referendarie universitarie con dinamica di scelta binaria (*Si/No*, Scheda Bianca, Scheda Nulla). Il sistema si interfaccia con i servizi di Identity Provider dell'Ateneo per l'autenticazione, ma applica protocolli di disaccoppiamento crittografico per garantire che l'identità civile del votante sia matematicamente separata dalla preferenza espressa, preservando i principi di anonimato, unicità e non-coercibilità.

1.2 Attori del Sistema

In conformità con la scomposizione dei privilegi volta a eliminare singoli punti di fallimento (*Single Point of Failure*), si identificano i seguenti attori onesti che operano nel sistema:

- **Studente (Elettore):** Il titolare del diritto di voto. Interagisce con il sistema per autenticarsi, richiedere un'autorizzazione anonima, comporre la scheda cifrata e verificare l'inclusione della stessa. Ha il vincolo stringente di non poter generare prove di sottomissione a terzi (Anti-coercizione).
- **Server di Autenticazione (Ateneo):** Il gestore dell'identità. Autentica lo studente tramite credenziali istituzionali (SSO), verifica lo stato di avente diritto e rilascia un'autorizzazione crittografica cieca. Non deve mai entrare in contatto con la scheda di voto o con il gettone in chiaro.
- **Urna Digitale (Tallier):** Il collettore delle schede. Raccoglie i pacchetti cifrati inviati dagli studenti, ne valida l'autorizzazione formale a urne aperte e, a urne chiuse, esegue il conteggio previa ricostruzione della chiave di decifratura. Non conosce l'identità degli inseritori.
- **Commissione Elettorale (Trustees):** Un insieme di n entità garanti (docenti, personale, studenti) che detengono in modo frazionato la capacità di abilitare la decifratura dell'urna. Nessun membro può operare singolarmente.

1.3 Funzionalità del Sistema

1.3.1 Requisiti Funzionali

- **Accreditamento e Gestione Identità:** Autenticazione forte degli utenti tramite SSO istituzionale e verifica dello stato di iscrizione attivo per la consultazione corrente.
- **Emissione delle Credenziali Anonime:** Generazione di un'attestazione di legittimità al voto che non esponga l'identità dell'elettore.
- **Sottomissione Cifrata della Preferenza:** Raccolta di schede elettorali cifrate in modo end-to-end dal client dello studente fino all'Urna Digitale.

- **Scrutinio Algoritmico Condizionato:** Decifratura e computazione dei risultati aggregati attivabile esclusivamente al raggiungimento del quorum crittografico dei Trustees.

1.3.2 Requisiti Non Funzionali

- **Trasparenza e Verificabilità:** Ogni fase dello scrutinio deve essere pubblicamente verificabile da controllori esterni senza esporre i dati sensibili.
- **Efficienza Computazionale Lato Client:** Le operazioni demandate al dispositivo dello studente (generazione chiavi effimere, accecamento, cifratura) devono presentare complessità polinomiale ridotta, compatibile con smart wallet o dispositivi mobili.
- **Decentralizzazione del Controllo:** Lo sblocco dei dati dell'urna non è subordinato a un server centrale, ma ripartito su base distributiva tra i Trustees.

1.4 Threat Models (Modello di Minaccia)

Si formalizzano le minacce e i vettori di attacco considerati nell'analisi di sicurezza del sistema. Per ciascun modello di minaccia si specifica il profilo dell'avversario in termini di capacità computazionali (PPT, *Probabilistic Polynomial-Time*) e di accesso alle risorse del sistema.

- TM.1 Iniezione e Falsificazione di Voti:** Un avversario PPT esterno o uno studente malizioso, privo di accesso privilegiato all'infrastruttura, tenta di inserire schede nell'urna senza autorizzazione o di duplicare voti legittimi (*Double Voting*). L'avversario ha accesso al canale di comunicazione pubblico verso l'Urna e può osservare i ciphertext in transito, ma non possiede la chiave privata del Server di Ateneo d né la chiave simmetrica effimera k_{sym} di un altro elettore.
- TM.2 De-anonimizzazione tramite Correlazione di Rete:** Un avversario PPT con ruolo di tecnico IT possiede accesso in lettura ai log di rete di entrambi i server (autenticazione e urna). Conosce i timestamp delle sessioni SSO e gli orari di ricezione dei pacchetti cifrati, ma non ha accesso al contenuto dei pacchetti né alle chiavi crittografiche. Il suo obiettivo è correlare statisticamente le due serie temporali per associare una matricola a una preferenza espressa.
- TM.3 Manomissione dell'Urna o della Commissione:** Un avversario con accesso amministrativo al database dell'Urna, oppure una coalizione di al più $t-1$ membri della Commissione (minoranza non raggiungente la soglia), tenta di alterare i record archiviati, eliminare voti validi o modificare le preferenze prima o durante lo scrutinio. L'avversario non possiede un numero sufficiente di quote di Shamir per ricostruire SK_{urna} autonomamente.
- TM.4 Coercizione e Vendita del Voto:** Un avversario (coercitore) con accesso fisico al dispositivo dell'elettore durante la sessione di voto tenta di costringerlo a esprimere una preferenza specifica e di ottenere una prova crittografica verificabile della scelta effettuata. Il coercitore può osservare lo schermo del dispositivo e leggere i valori visualizzati, ma non ha accesso ai log interni del client né al gettone segreto m memorizzato dal wallet.

TM.5 Interruzione e Crash dello Scrutinio: Un attacco Denial of Service (DoS) o un guasto hardware durante la fase di tallying causa la perdita dei dati volatili o l'impossibilità di ricostruire lo stato del sistema. L'avversario non modifica i dati persistiti su disco, ma interrompe il processo di elaborazione in memoria.

TM.6 Ripudio del Voto: Uno studente che ha espresso una preferenza tenta successivamente di negare di averla sottomessa, affermando di non aver partecipato al voto o che la propria scheda è stata inserita da terzi a sua insaputa. L'avversario è il legittimo titolare della credenziale di voto (m, s) .

1.5 Proprietà del Sistema da Preservare Sotto Attacco

L'architettura crittografica è progettata per garantire le seguenti proprietà di sicurezza stabili:

- **Anonimato (Confidenzialità):** Separazione matematica tra l'identità dello studente e la scheda depositata.
- **Integrità End-to-End:** Inalterabilità del voto dal momento della generazione sul client fino al calcolo matematico del tally.
- **Unicità del Voto:** Assicurazione che ogni gettone autorizzato pesi esattamente uno nel computo finale, scartando i tentativi di replay.
- **Receipt-Freeness:** Impossibilità per lo studente di produrre una ricevuta crittografica che dimostri a terzi la preferenza espressa, mitigando i tentati attacchi di coercizione.
- **Resilienza Operativa:** Capacità di completare lo scrutinio anche in caso di indisponibilità di una quota minoritaria di membri della Commissione.
- **Non Ripudiabilità Limitata:** La partecipazione al voto (ma non la preferenza espressa) è verificabile pubblicamente tramite il Bulletin Board, rendendo impossibile negare di aver sottomesso una scheda una volta che il gettone sia stato pubblicato tra i voti convalidati.

2 WP2: Progettazione della Soluzione

2.1 Strategia di Generazione e Distribuzione delle Chiavi

La radice di fiducia e la separazione dei compiti si basano su una specifica configurazione di Infrastruttura a Chiave Pubblica (PKI) e schemi di suddivisione del segreto:

1. **PKI di Ateneo (Auth Server):** Il server di autenticazione genera una coppia di chiavi asimmetriche RSA stabili mediante l'algoritmo $Gen_{RSA}(1^\lambda)$. La chiave pubblica (e, n) viene pubblicata sulla directory d'Ateneo ed è cablata nel software client. La chiave privata d viene isolata all'interno di un modulo hardware sicuro (HSM) e utilizzata esclusivamente per apporre firme cieche.
2. **Chiave dell'Urna e Secret Sharing di Shamir:** L'Urna Digitale, in fase di setup dell'elezione, inizializza una coppia di chiavi asimmetriche (es. RSA o ECC). La chiave pubblica PK_{urna} viene distribuita pubblicamente per consentire la cifratura dei voti. La chiave privata SK_{urna} costituisce il segreto fondamentale per l'apertura dei voti; essa viene frammentata tramite uno **Schema di Shamir** (t, n) e distribuita agli n membri della Commissione Elettorale.

Formalmente, si definisce un polinomio casuale di grado $t - 1$ su un campo finito $\mathbb{Z}_p$:

$$f(x) = SK_{urna} + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1} \pmod{p}$$

Ogni coordinatore i della commissione riceve la quota segreta (x_i, y_i) dove $y_i = f(x_i)$. Al termine della distribuzione, la chiave originale SK_{urna} viene permanentemente sovrascritta e distrutta dalla memoria volatile dell'urna tramite istruzioni esplicite di *memory zeroization*. La soglia viene impostata a $t = \lceil 2/3 \cdot n \rceil$.

Nota sulle Assunzioni di Setup: Il presente modello assume l'Urna come *trusted* esclusivamente nella limitata fase iniziale di setup. In scenari ad altissima criticità, per eliminare il *Single Point of Failure* rappresentato dall'istante in cui l'Urna possiede in memoria la chiave SK_{urna} non ancora frammentata, è possibile ricorrere a protocolli di **Distributed Key Generation (DKG)**. In tale configurazione alternativa, sarebbero i membri della Commissione a generare congiuntamente le quote y_i e la PK_{urna} pubblica, senza che il segreto intero venga mai elaborato in chiaro su un singolo nodo.

2.2 Identificazione dello Studente e Fase di Registrazione (Blind Signature con FDH)

L'identificazione avviene tramite l'infrastruttura di Single Sign-On (SSO) dell'Ateneo. Per prevenire la falsificazione di credenziali basata sull'omomorfismo moltiplicativo intrinseco dell'algoritmo RSA (vulnerabilità del *Textbook RSA*), il sistema implementa lo schema **RSA-FDH (Full Domain Hash)** abbinato al protocollo di **Firma Cieca di Chaum**:

1. **Generazione e Blinding (Client):** Il dispositivo dello studente genera un gettone identificativo univoco $m \in \{0, 1\}^{128}$ tramite un CSPRNG. Prima dell'acceccamento, calcola l'hash crittografico del gettone $H(m)$ mappato sul dominio di RSA. Successivamente, seleziona un fattore di acceccamento casuale $r \in \mathbb{Z}_n^*$ tale che $\gcd(r, n) = 1$. Il client calcola il gettone accecato:

$$m' = H(m) \cdot r^e \pmod{n}$$

Il valore m' viene trasmesso al Server di Autenticazione unitamente al token di sessione SSO.

2. **Signing (Server):** Il Server verifica che la matricola sia presente nelle liste degli aventi diritto e che il flag `has_voted` sia falso. Se i controlli hanno esito positivo, il server imposta `has_voted = true` e calcola la firma cieca:

$$s' = (m')^d \pmod{n}$$

Il valore s' viene restituito al client.

3. **Unblinding (Client):** Lo studente riceve s' ed estrae la firma pulita moltiplicandola per l'inverso modulare del fattore di accecamento:

$$s = s' \cdot r^{-1} = (H(m) \cdot r^e)^d \cdot r^{-1} \equiv H(m)^d \cdot r \cdot r^{-1} \equiv H(m)^d \pmod{n}$$

La coppia (m, s) rappresenta la credenziale anonima di voto. La validità è verificabile da chiunque mediante la chiave pubblica dell'Ateneo controllando che $s^e \equiv H(m) \pmod{n}$. L'uso della funzione hash previene gli attacchi esistenziali basati sulla proprietà omomorfa del Textbook RSA.

2.3 Formato dei Messaggi e della Scheda di Voto

2.3.1 Struttura Dati (JSON)

La scheda S è formattata come oggetto JSON contenente i seguenti campi strutturati:

- **preferenza:** Stringa vincolata ai valori di dizionario ["SI", "NO", "BIANCA", "NULLA"].
- **gettone_m:** Stringa esadecimale rappresentante il gettone m a 128-bit.
- **firma_s:** Stringa esadecimale contenente la firma de-blinded s .
- **timestamp:** Marcatura temporale estesa in formato standard ISO 8601.
- **h_bind:** Hash di binding calcolato come:

$$\text{h_bind} = \text{SHA-256}(\text{preferenza} \parallel \text{gettone_m} \parallel \text{timestamp})$$

L'introduzione dell'hash di binding (**h_bind**) salda la scelta di voto al gettone e all'istante temporale, impedendo ad attori interni (es. gestori dell'urna) di modificare la preferenza alterando il payload JSON senza corrompere la validità dell'hash stesso.

```

{
  "preferenza": "SI" | "NO",
  "gettone_m": "valore_m",
  "firma_s": "valore_s",
  "timestamp": "2026-05-20T14:30:05Z"
}

```

Figura 1: Formato logico e serializzazione della scheda di voto (JSON)

2.3.2 Operazione di Cifratura (Schema Ibrido AEAD)

Per superare i limiti di dimensione del payload imposti da RSA-OAEP e garantire la resistenza contro attacchi al testo cifrato scelto (CCA-security), viene implementato uno schema di cifratura autenticata ibrida (KEM/DEM). Per evitare manipolazioni silenziose della chiave asimmetrica, il ciphertext della chiave viene incluso come *Associated Data* (AAD) nel calcolo del tag simmetrico:

1. Il client genera una chiave simmetrica effimera $k_{sym} \xleftarrow{\$} \{0,1\}^{256}$ e un Vettore di Inizializzazione crittograficamente sicuro $iv \xleftarrow{\$} \{0,1\}^{96}$. Poiché k_{sym} è generata ex-novo per ogni scheda, il rischio di riutilizzo del Nonce è strutturalmente nullo.
2. La chiave simmetrica viene cifrata con la chiave pubblica dell'Urna mediante RSA-OAEP, ottenendo l'incapsulamento della chiave:

$$C_{key} \leftarrow \text{RSA-OAEP}_{PK_{urna}}(k_{sym})$$

3. Il corpo del JSON della scheda S viene cifrato tramite algoritmo AES-256 in modalità GCM. Si utilizza C_{key} come dato associato (*AAD*) per legare indissolubilmente la parte simmetrica a quella asimmetrica:

$$(C_{sym}, \tau) \leftarrow \text{AES-GCM}_{k_{sym}, iv}(S, \text{AAD} = C_{key})$$

4. Il messaggio finale trasmesso all'urna è la stringa combinata: $C = (iv \parallel C_{key} \parallel C_{sym} \parallel \tau)$.

2.4 Flusso di Comunicazione e Registrazione del Voto

Il flusso operativo applica contromisure specifiche contro i vettori di attacco di analisi del traffico temporale (*Timing Attacks*):

1. **Preparazione ed Introduzione del Jitter:** Il client compone la scheda S , calcola la cifratura ibrida C . Per sventare l'analisi dei log incrociati da parte del *Tecnico IT* (TM.2), il client non invia il pacchetto immediatamente dopo la firma cieca, ma introduce un ritardo stocastico forzato (*jittering*) calcolato casualmente in un intervallo $\Delta t \in [1, 5]$ minuti, sufficiente a rompere l'allineamento statistico senza compromettere l'esperienza utente.

2. **Trasmissione Anonima:** Il pacchetto C viene trasmesso all'Urna Digitale intradando i pacchetti attraverso una rete anonimizzante (es. rete Tor o mix-net d'Ateneo) per mascherare l'indirizzo IP di origine.
3. **Batching e Registrazione sull'Urna:** L'Urna Digitale riceve i pacchetti e valida l'integrità strutturale del ciphertext simmetrico verificando il tag τ . I pacchetti validi non vengono memorizzati nell'ordine esatto di arrivo: l'urna gestisce code di accumulo orarie (*batching*), esegue un rimescolamento casuale (*shuffling*) dei blocchi ricevuti e solo successivamente esegue la persistenza sul database ad architettura *append-only*. L'urna rilascia come conferma l'hash $\text{SHA-256}(C)$.

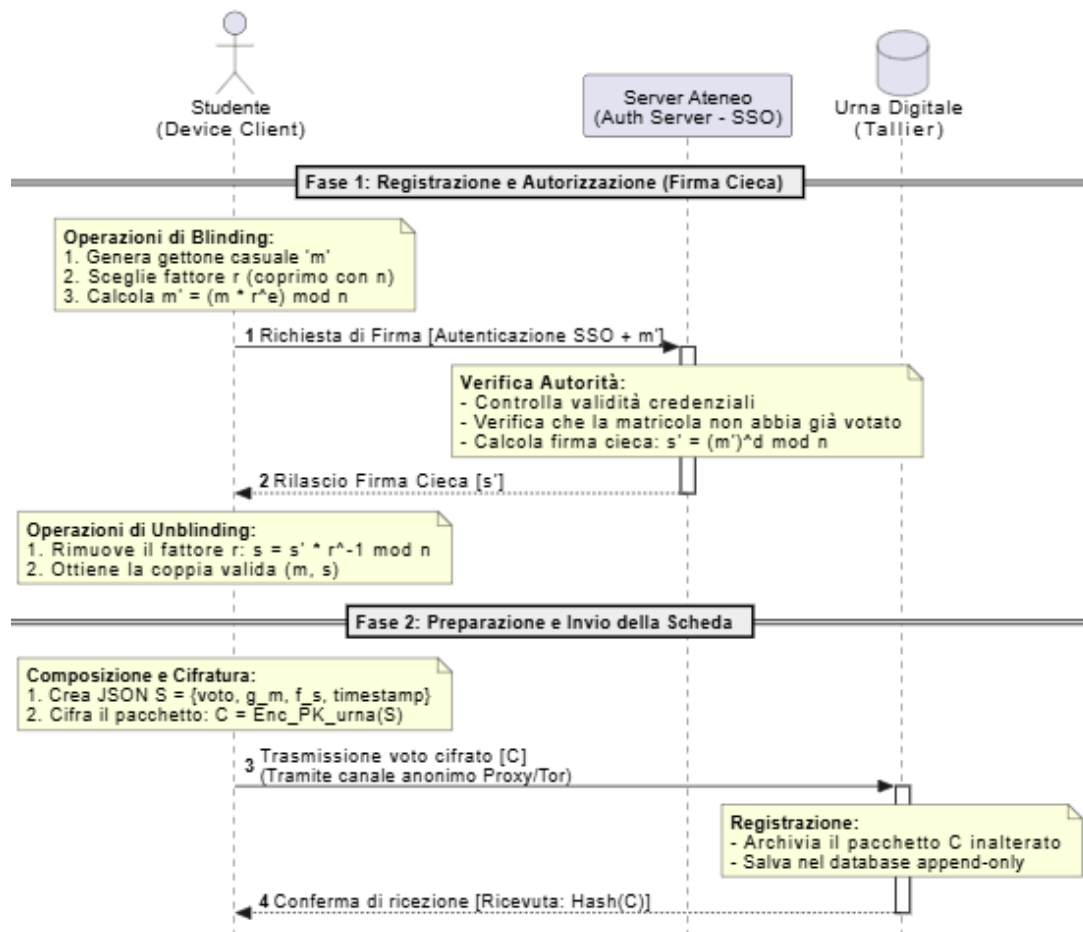


Figura 2: Diagramma di sequenza del flusso di autenticazione, blinding e sottomissione del voto

2.5 Meccanismo di Sostituzione o Annullamento

Per garantire l'assenza di ricevuta probante (*Receipt-Freeness*), il sistema supporta la sovrascrittura del voto:

- **Sostituzione:** Uno studente soggetto a coercizione può inviare una nuova scheda cifrata C_{new} riutilizzando lo stesso identico gettone m e la firma s , ma inserendo un valore di `timestamp` aggiornato.

- **Logica di Risoluzione:** L'Urna Digitale accetta e memorizza tutti i pacchetti in arrivo associati allo stesso gettone m . La logica di deduplicazione opererà in fase di scrutinio mantenendo esclusivamente il record associato al timestamp cronologicamente più recente, invalidando i precedenti.

2.6 Procedura di Scrutinio e Resilienza Operativa

Al termine della finestra temporale di votazione, l'Urna interrompe l'accettazione dei pacchetti di rete e avvia la fase di calcolo:

1. **Ricostruzione Idempotente del Segreto:** I membri della commissione si autenticano e presentano le proprie quote segrete (x_i, y_i) . Raggiunta la soglia di sicurezza t , il sistema esegue l'interpolazione di Lagrange per ricalcolare il polinomio nel punto zero:

$$SK_{urna} = f(0) = \sum_{i=1}^t y_i \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \pmod{p}$$

Resilienza Operativa: Per mitigare crash hardware del server durante lo scrutinio (TM.5), il software di tallying implementa una logica di sottomissione idempotente. Le chiavi parziali caricate non modificano lo stato del database e, in caso di crash della memoria volatile, la procedura può essere rieseguita immediatamente reinserendo le medesime quote, senza richiedere una rigenerazione dei parametri.

2. **Decifratura, Filtraggio e Validazione:** L'Urna utilizza SK_{urna} per estrarre la chiave simmetrica k_{sym} da C_{key} , decifra il corpo della scheda mediante AES-GCM (utilizzando iv trasmesso in chiaro nel pacchetto C) e convalida i requisiti logici:
 - Verifica che la firma dell'Ateneo sia valida: l'Urna legge il campo `gettone_m` dalla scheda decifrata, ottiene m , calcola $H(m)$ e controlla che $s^e \equiv H(m) \pmod{n}$.
 - Verifica la congruenza dell'hash di binding: $h_bind \equiv \text{SHA-256}(\text{preferenza} \parallel \text{gettone_m} \parallel \text{timestamp})$.
 - Esegue la deduplicazione: se un gettone m compare in più schede decifrate, viene isolata solo la scheda con il timestamp più recente.
3. **Aggregazione:** Le preferenze delle schede convalidate vengono sommate nei registri dei risultati finali.

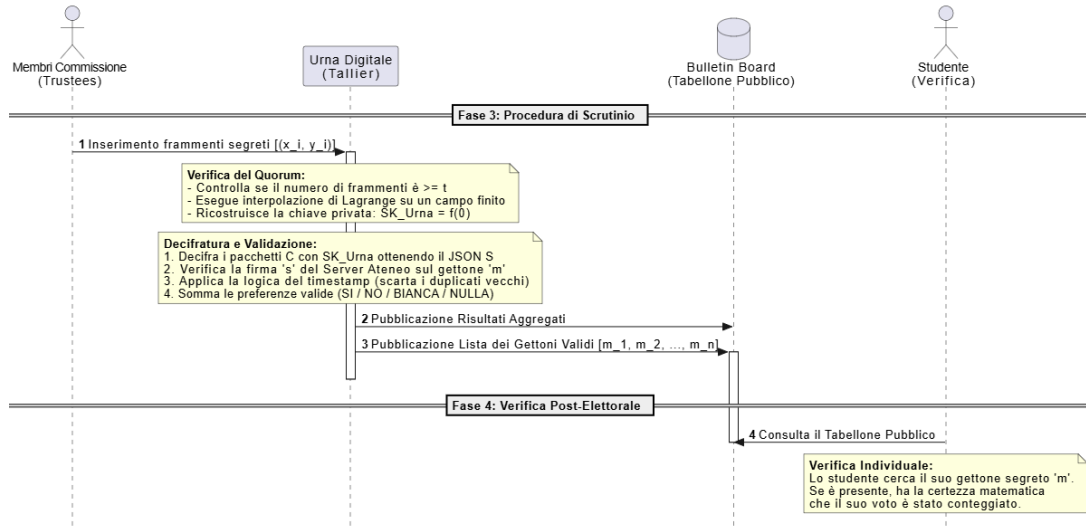


Figura 3: Diagramma di sequenza della fase di scrutinio, tallying e auditing post-elettorale

2.7 Meccanismo di Verifica Individuale e Universale (Bulletin Board Autenticato)

Al fine di garantire che l'ente gestore dell'urna non possa omettere voti legittimi o inserire voti fittizi senza essere rilevato, il sistema implementa un **Bulletin Board strutturato come Albero di Merkle (Merkle Tree)**:

- **Struttura del Tabellone:** I gettoni m_i estratti dalle sole schede convalidate e conteggiate vengono ordinati e posizionati come nodi foglia di un albero di hash binario. L'Urna calcola ricorsivamente gli hash intermedi fino ad ottenere una radice unica (*Merkle Root*). La Merkle Root viene firmata digitalmente con RSA-PSS utilizzando la **chiave privata dell'Urna** (SK_{urna}) ricostruita dai Trustee durante lo scrutinio — la stessa chiave impiegata per la decifratura delle schede. Nel modello, la Commissione Elettorale opera *attraverso* l'Urna: sono i Trustee che forniscono le quote Shamir per ricostruire SK_{urna} , e la firma della Merkle Root avviene all'interno della procedura di scrutinio (**tally()**), congelando la struttura dati in modo immutabile.
- **Verifica Individuale:** L'Urna pubblica il Merkle Tree e fornisce ad ogni client la relativa *Merkle Proof* (l'insieme dei nodi fratelli necessari per risalire dalle foglie alla root). Lo studente inserisce il proprio gettone segreto m nel modulo di verifica: il client calcola il percorso di hash e verifica la corrispondenza con la Root firmata. Se l'hash coincide, vi è la certezza matematica che il voto è stato incluso nel computo.
- **Verifica Universale:** Qualsiasi entità esterna può scaricare l'intero Bulletin Board e verificare in modo indipendente l'elezione eseguendo tre controlli matematici:
 - (i) Verifica la firma digitale RSA-PSS sulla Merkle Root usando la chiave pubblica dell'Urna (PK_{urna});
 - (ii) Verifica che ogni gettone m_i presente nelle foglie sia effettivamente valido, controllando la firma ad esso associata ($s_i^e \equiv H(m_i) \pmod{n}$);
 - (iii) Verifica l'assenza di duplicati foglia e ricalcola l'albero per controllare che la somma aritmetica dei voti corrisponda al risultato aggregato proclamato.

Nota sulla Privacy: La pubblicazione in chiaro dei gettoni m_i espone il fatto che un utente ha partecipato al voto, ma non rivela in alcun modo la preferenza espressa, mantenendo intatta la riservatezza della scheda.

3 WP3: Analisi di Sicurezza e Mitigazione delle Minacce

3.1 Obiettivo dell'Analisi

In questa sezione si dimostra formalmente la resilienza dell'architettura proposta rispetto alle minacce modellate nel WP1. L'analisi valuta come le primitive crittografiche e i protocolli di rete definiti nel WP2 garantiscano le proprietà di sicurezza richieste, operando sotto l'assunzione standard che gli algoritmi sottostanti (RSA, AES, SHA-256) siano computazionalmente sicuri.

3.2 Analisi delle Proprietà Crittografiche

3.2.1 Anonimato e Unlinkability

L'impossibilità di tracciare il legame tra identità civile ed espressione di voto è garantita dalla **Firma Cieca di Chaum con FDH**. Il Server di Ateneo firma il messaggio accecato $m' = H(m) \cdot r^e \pmod{n}$, senza poter mai osservare il gettone m in chiaro. Poiché il fattore r è scelto casualmente dallo studente in $\mathbb{Z}_n^*$, l'operazione di *unblinding* restituisce una coppia (m, s) matematicamente valida ma statisticamente indipendente da m' . Pertanto, anche nel caso peggiore in cui il Server di Ateneo e l'Urna Digitale dovessero colludere (condividendo i log di rete), non esisterebbe alcuna correlazione algebrica per risalire alla matricola partendo dal gettone.

3.2.2 Integrità e CCA-Security

L'integrità della singola scheda in transito è assicurata dal paradigma **AEAD (AES-GCM)**. La modalità *Galois/Counter Mode* calcola un tag di autenticazione τ sul testo cifrato, protetto dall'uso di un Nonce generato casualmente ad ogni trasmissione. Qualsiasi manipolazione anche di un singolo bit provocherà il fallimento della validazione del tag prima della decifrazione, proteggendo l'urna da attacchi attivi al testo cifrato scelto (*CCA-Security*) e attacchi di tipo *Padding Oracle*. L'integrità logica della preferenza è invece blindata dall'hash interno `h_bind`. Se un attaccante interno tentasse di alterare il campo `preferenza` in chiaro, il binding crittografico fallirebbe in fase di tallying:

$$\text{SHA-256}(\text{preferenza_alterata} \parallel \text{gettone} \parallel \text{timestamp}) \neq \text{h_bind}$$

3.2.3 Anti-Coercizione e Receipt-Freeness

Il protocollo è progettato per non fornire alcuna ricevuta che dimostri in modo inconfutabile la preferenza a un terzo. Se un coercitore obbligasse lo studente a votare in sua presenza, lo studente potrebbe, non appena al sicuro, generare un nuovo pacchetto C_{new} riutilizzando lo stesso gettone m , ma con un timestamp superiore e preferenza opposta. L'urna terrà valido solo il voto più recente.

Assunzioni critiche del modello: Questo meccanismo si regge su alcune assunzioni fondamentali. Primo, l'elettore deve avere accesso a un momento di privacy prima della chiusura delle urne. Secondo, **il gettone m deve restare rigorosamente segreto**: se il coercitore riuscisse a memorizzare o registrare il valore di m durante la sessione coercitiva, potrebbe in seguito consultare il Bulletin Board pubblico, compromettendo la *receipt-freeness* e verificando se il voto conteggiato per quel gettone è effettivamente quello

imposto. Infine, si assume l'impraticabilità su larga scala di attacchi di tipo *Midnight Voting*, in cui il coercitore impone il voto negli istanti immediatamente precedenti la chiusura delle urne, negando all'elettore la finestra temporale materiale necessaria per effettuare la sovrascrittura della propria preferenza.

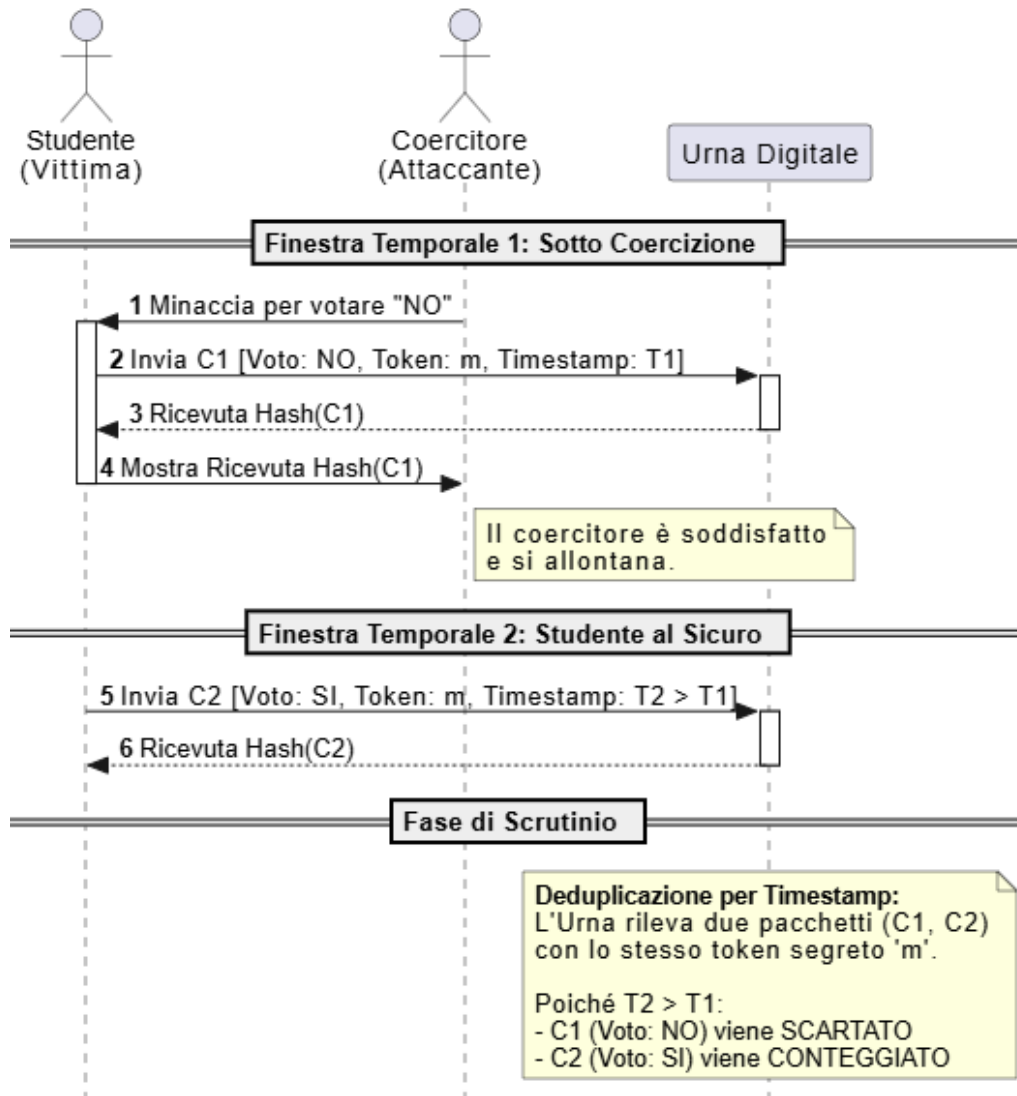


Figura 4: Meccanismo di sovrascrittura del voto per garantire la Receipt-Freeness.

3.3 Risoluzione dei Threat Models (TM)

Di seguito si mappa in modo analitico la mitigazione crittografica per ogni vettore di attacco identificato nel Modello di Minaccia (§1.4):

- **TM.1 - Iniezione e Falsificazione (Double Voting):** Forgiare un voto falso richiede di calcolare $s = H(m)^d \pmod{n}$ senza conoscere d , un'operazione resa intrattabile dalla durezza del problema RSA. L'uso dell'hashing (RSA-FDH) scongiura attacchi di tipo esistenziale o moltiplicativo che affliggono il Textbook RSA: non è possibile combinare firme valide su gettoni distinti per forgiarne una terza. I tentativi di *Replay Attack* vengono bloccati su due livelli distinti. A livello di rete, il controllo sull'hash $\text{SHA-256}(C)$ funge da prima barriera (ottimizzazione) per

scartare pacchetti identici. Tuttavia, **la vera garanzia crittografica contro il doppio voto risiede a livello logico (post-decifratura)**: è l'unicità del gettone m a forzare la deduplicazione sul tabellone. Anche se un attaccante costruisse un pacchetto completamente nuovo e crittograficamente valido contenente un gettone m già usato, il sistema conterebbe sempre e solo una singola preferenza per quell' m .

- **TM.2 - De-anonimizzazione tramite Timing Attack:** La minaccia del Tecnico IT viene neutralizzata dal disaccoppiamento temporale. L'impiego del **Jittering stocastico** ($\Delta t \in [1, 5]$ minuti) lato client distrugge l'allineamento deterministico tra i log SSO e il momento di invio in rete. A valle, il meccanismo di **Batching e Shuffling** dell'Urna disaccoppia ulteriormente l'orario di ricezione del pacchetto (livello di rete) dal suo indice di memorizzazione (livello di persistenza), rendendo inefficace l'analisi statistica del traffico.

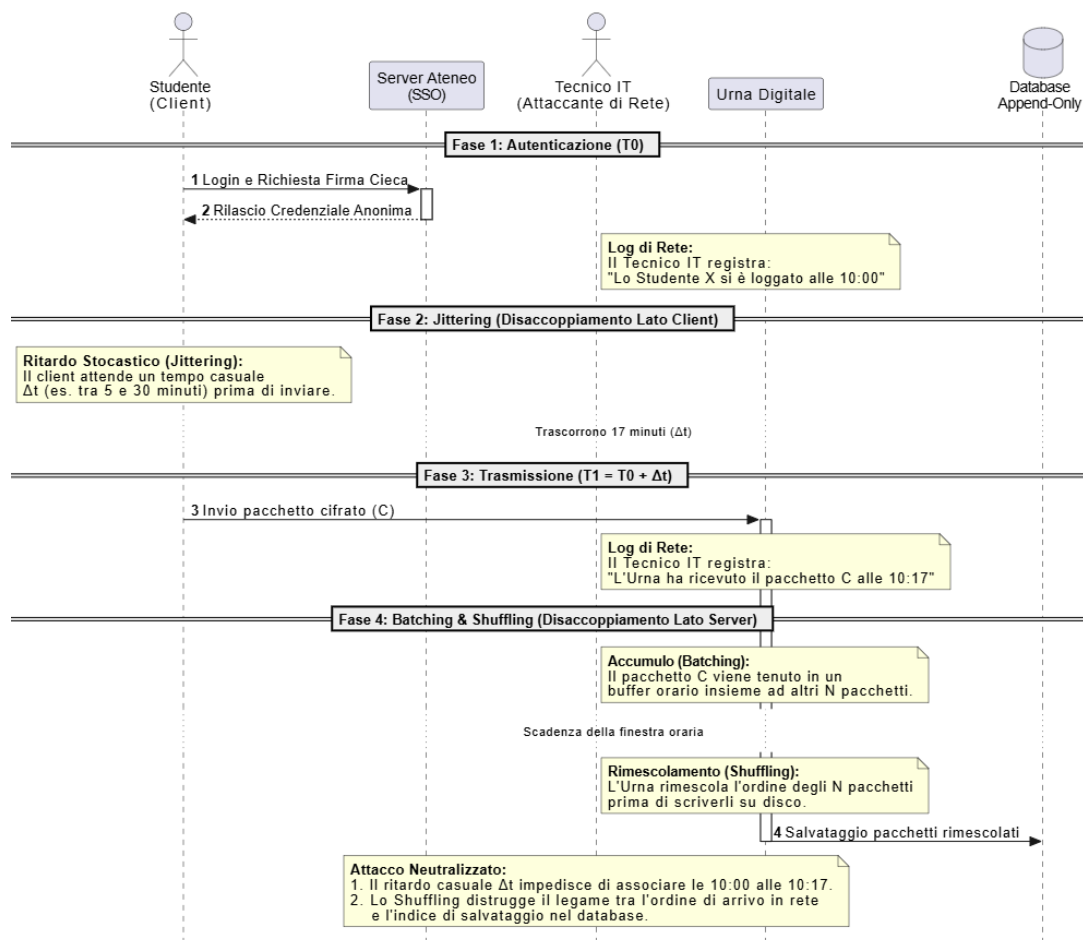


Figura 5: Disaccoppiamento temporale per la mitigazione dei Timing Attack.

- **TM.3 - Manomissione Urna o Commissione:** Una fazione minoritaria della Commissione non può forzare l'apertura anticipata dell'urna poiché il calcolo del polinomio di Lagrange richiede strettamente il raggiungimento della soglia t . Inoltre, l'Urna non può cancellare o omettere schede valide prima dello scrutinio grazie al **Bulletin Board autenticato**. L'omissione di un gettone m provocherebbe il fallimento immediato della *Merkle Proof* lato studente. *Modello di verifica:* Questa garanzia matematica è effettiva a patto che esista un sottoinsieme di elettori

motivati a eseguire la verifica individuale; è sufficiente che anche un solo studente denunci il fallimento della propria Merkle Proof per esporre in modo inconfutabile e crittografico la manomissione dell'Urna.

- **TM.4 - Coercizione e Vendita del Voto:** Come discusso nel paragrafo 3.2.3, la sovrascrittura incondizionata del voto in finestre temporali successive rende logicamente ed economicamente impraticabile l'acquisto della preferenza. Sotto la stretta assunzione che il gettone m non venga esposto visivamente al coercitore e dell'assenza di attacchi *Midnight Voting*, quest'ultimo non avrà mai alcuna garanzia che il voto espresso sotto minaccia non venga successivamente annullato o modificato dal legittimo titolare.
- **TM.5 - Interruzione e Crash dello Scrutinio:** L'eventualità di un guasto hardware, perdita di stato o attacco Denial of Service (DoS) durante la delicata unione dei segreti di Shamir è gestita dalla logica di **Idempotenza**. Poiché la ricostruzione di SK_{urna} e la verifica di τ avvengono come funzioni pure senza mutare irreversibilmente le quote fornite in input, il processo di scrutinio può essere ripetuto *ex-novo* al ripristino dell'infrastruttura, garantendo la totale continuità e resilienza operativa del referendum.
- **TM.6 - Ripudio del Voto:** Uno studente che tenti di negare la propria partecipazione al voto è confutato dalla pubblicazione pubblica del proprio gettone m_i sul Bulletin Board. Chiunque può verificare che il gettone sia presente nell'albero di Merkle e che la firma associata s_i sia valida rispetto alla chiave pubblica dell'Ateneo ($s_i^e \equiv H(m_i) \pmod{n}$). Poiché il gettone m è stato generato dal client dello studente stesso e la firma è stata rilasciata dall'Ateneo solo dopo l'autenticazione SSO, la copresenza di m_i nel Bulletin Board firmato e del record `has_voted = true` nel database dell'Ateneo costituisce evidenza non ripudiabile della partecipazione. Si noti tuttavia il trade-off con l'anonimato: questa forma limitata di non ripudiabilità riguarda esclusivamente la *partecipazione* al voto, non la preferenza espressa, che rimane protetta dalla separazione crittografica garantita dalla Firma Cieca.

3.4 Analisi delle Alternative Progettuali e Trade-off

Nel processo di progettazione dell'architettura descritta nel WP2, sono state valutate diverse alternative per le primitive e i protocolli crittografici impiegati. Di seguito si motivano le scelte effettuate rispetto alle opzioni scartate, mostrando che non esistono modifiche che migliorino una proprietà senza introdurre perdite altrove.

Firma Cieca vs. Zero-Knowledge Proof per l'autorizzazione anonima. Un'alternativa alla Firma Cieca di Chaum sarebbe l'impiego di prove a conoscenza zero (ZKP), in particolare schemi come zk-SNARKs, per consentire allo studente di dimostrare di essere un avente diritto senza rivelare la propria identità. Questa soluzione offrirebbe garanzie di anonimato più forti in alcuni scenari avanzati. Tuttavia, il costo computazionale lato client per la generazione di una prova zk-SNARK è di diversi ordini di grandezza superiore rispetto alle operazioni di blinding RSA, rendendo la soluzione incompatibile con il requisito non funzionale di efficienza su dispositivi mobili (§1.3.2). La Firma Cieca di Chaum, combinata con RSA-FDH, offre garanzie di unlinkability equivalenti nel modello di minaccia considerato a un costo computazionale polinomiale ridotto.

Shamir Secret Sharing vs. Threshold Encryption per la chiave dell'urna. In luogo dello schema di Shamir per frammentare SK_{urna} , sarebbe possibile adottare

uno schema di *threshold encryption* (es. ElGamal a soglia), in cui i voti vengono cifrati direttamente con la chiave pubblica del gruppo e ogni Trustee contribuisce parzialmente alla decifratura senza che SK_{urna} venga mai ricostruita su un singolo nodo. Questo eliminerebbe il Single Point of Failure temporaneo nella fase di setup discusso in §2.1. Tuttavia, questa architettura richiederebbe un'interazione multi-round tra i Trustees durante lo scrutinio e l'adozione di librerie crittografiche più complesse e meno consolidate nel panorama dei tool standard. Data la natura del contesto applicativo (referendum universitario, non elezioni nazionali), si è ritenuto che il compromesso introdotto da Shamir — accettabile a fronte di una fase di setup sicura e di una *memory zeroization* immediata — fosse preferibile rispetto alla complessità implementativa aggiuntiva. Il modello DKG rimane esplicitamente documentato come upgrade percorribile (§2.1).

Deduplicazione per timestamp vs. meccanismo di revoca esplicita. La sovrascrittura del voto tramite timestamp aggiornato è un meccanismo semplice e privo di stato lato server. Un'alternativa sarebbe un protocollo di revoca esplicita, in cui l'elettore firma un messaggio di annullamento e il server lo registra separatamente. Questa soluzione offrirebbe una semantica più chiara, ma richiederebbe che il server di revoca conosca il gettone m — violando il principio di unlinkability che costituisce il nucleo della Firma Cieca. La deduplicazione per timestamp, operando interamente in fase di scrutinio post-decifratura, preserva l'anonimato del flusso di rete a costo di una finestra di vulnerabilità al Midnight Voting già discussa in §3.2.3.

Bulletin Board con Merkle Tree vs. trasparenza omomorfica. Per la verificabilità universale, un'alternativa è l'uso di schemi di cifratura omomorfica che consentano di verificare l'aggregazione dei voti senza decifrarli individualmente. Questa tecnica, impiegata in sistemi come Helios v3, garantisce che nemmeno i Trustees conoscano le preferenze individuali al momento del conteggio. Nel nostro scenario, tuttavia, i voti vengono decifrati in chiaro durante lo scrutinio e la riservatezza individuale è già garantita dall'anonimato del gettone. Introdurre la cifratura omomorfica avrebbe aggiunto complessità computazionale significativa (sia per la generazione lato client che per lo scrutinio) senza apportare benefici addizionali rispetto alle proprietà di segretezza già conseguite con l'approccio ibrido AEAD.

4 WP4: Implementazione e Prestazioni

4.1 Ambiente di Sviluppo e Tecnologie Utilizzate

Il sistema progettato nel WP2 è stato implementato come **applicazione web standalone** eseguibile su un singolo computer, che simula l'intera infrastruttura elettorale (Ateneo, Urna, Bacheca pubblica e dispositivi degli elettori) all'interno di un unico processo Flask.

4.1.1 Stack Tecnologico

Componente	Tecnologia / Versione
Linguaggio	Python 3.12
Framework Web	Flask ≥ 3.0
Libreria crittografica	<code>cryptography</code> (hazmat layer) ≥ 44.0
Autenticazione OIDC	<code>Authlib</code> ≥ 1.4
Formato dati	JSON

Tabella 1: Stack Tecnologico impiegato nel prototipo

La scelta di Python con la libreria `cryptography` consente l'accesso diretto alle **primitive a basso livello** (hazmat layer) necessarie per implementare lo schema RSA-FDH di Blind Signature, mantenendo al contempo API sicure e ben documentate per le operazioni standard (RSA-OAEP, AES-GCM, SHA-256).

4.2 Architettura del Prototipo

Il sistema è composto da **quattro entità logiche** che corrispondono ai ruoli definiti nel WP2:

- **Authentication Server** (`auth_server.py`): Simula l'Ateneo e il suo HSM. All'avvio genera una coppia di chiavi RSA a 2048 bit per la firma cieca. Mantiene un registro degli aventi diritto e previene il Double Voting. Esegue la firma cieca $s' = (m')^d \pmod{n}$.
- **Client Wallet** (`client_wallet.py`): Rappresenta il dispositivo dell'elettore. Gestisce la generazione del gettone m (128 bit), il Full-Domain Hash (FDH), il blinding con fattore r , l'unblinding e la cifratura ibrida KEM/DEM della scheda.
- **Urna Elettorale** (`urna.py`): Gestisce la coppia RSA dell'Urna e lo Shamir Secret Sharing ($t=2, n=3$) per proteggere la chiave master AES-256. Accoda i pacchetti cifrati e gestisce lo scrutinio tramite interpolazione di Lagrange, validando e deduplicando i voti.
- **Bacheca Pubblica** (`bacheca.py`) e **IDP**: Costruisce il Merkle Tree (SHA-256) sulle foglie. Include l'Identity Provider OIDC simulato (`mock_oidc_server.py`) per l'emissione di ID Token JWT.

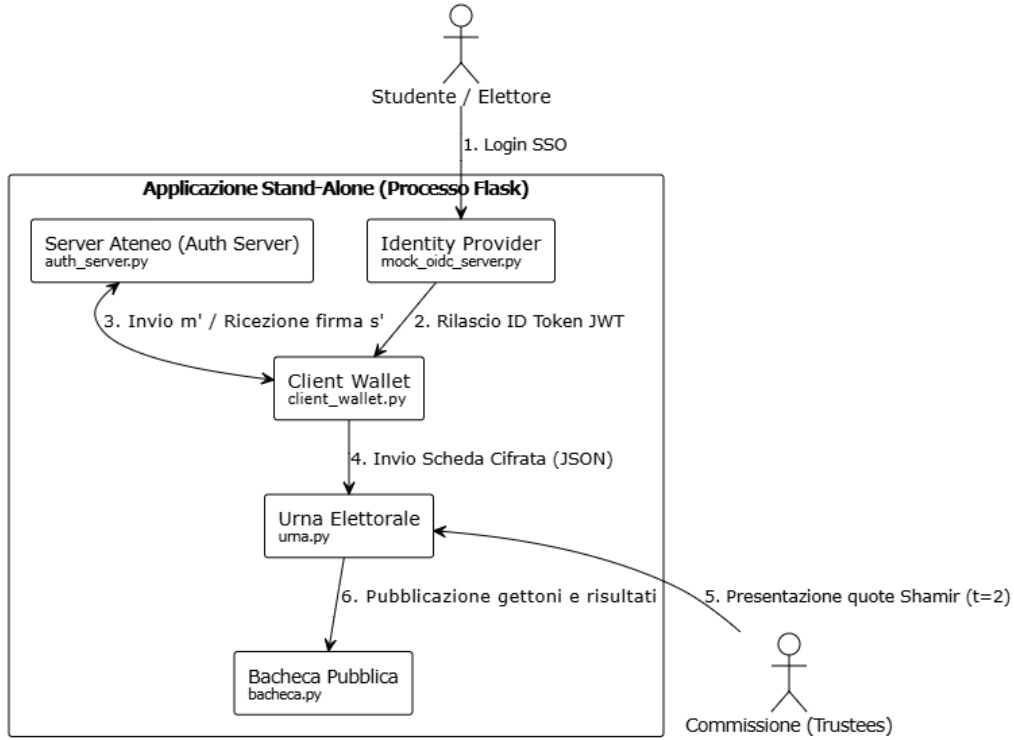


Figura 6: Schema logico dell'architettura del prototipo stand-alone

4.3 Flusso del Protocollo Implementato

Il protocollo si articola nelle seguenti fasi:

1. **Fase 1 — Autenticazione dell'Elettore:** L'elettore accede tramite SSO OIDC.
2. **Fase 2 — Emissione della Credenziale Anonima:** Il Client genera m e calcola m' . Il Server verifica e calcola s' . Il Client esegue l'unblinding ottenendo (m, s) .
3. **Fase 3 — Cifratura e Invio della Scheda:** Il payload JSON viene costruito includendo h_bind . La cifratura avviene con schema KEM (RSA-OAEP) / DEM (AES-256-GCM), usando il KEM come AAD per vincolare crittograficamente il pacchetto. Nel prototipo, il jitter temporale è simulato come no-op.
4. **Fase 4 — Scrutinio con Quorum:** I Trustee presentano $t = 2$ quote. SK_{urna} viene ricostruita. I pacchetti subiscono *shuffling*, vengono decifrati e validati (firma cieca e h_bind). Segue la deduplicazione posticipata.
5. **Fase 5 — Pubblicazione e Verifica:** I voti validati finiscono nella Bacheca. Viene costruito il Merkle Tree, e la Root viene firmata con RSA-PSS usando SK_{urna} .

4.4 Primitive Crittografiche Impiegate

Primitiva	Parametri	Scopo / Modulo
RSA-FDH	2048 bit, SHA-256 iterativo	Blind Signature (<code>auth_server.py</code>)
RSA-OAEP	2048 bit, SHA-256, MGF1	KEM per incapsulamento chiave (<code>client_wallet.py</code>)
RSA-PSS	2048 bit, SHA-256	Firma Merkle Root con SK_{urna} (<code>urna.py</code>)
AES-256-GCM	256 bit, nonce 96 bit	DEM su JSON, protezione Key Urna
Shamir SSS	$t=2, n=3, p = 2^{256} + 297$	Distribuzione master key (<code>urna.py</code>)
SHA-256	256 bit	Hash FDH, Merkle Tree, h_bind
JWT RS256	RSA-2048	ID Token OIDC (<code>mock_oidc_server.py</code>)

Tabella 2: Riassunto delle primitive crittografiche implementate

4.5 Dimensione dei Messaggi Scambiati

Il pacchetto cifrato C scambiato sulla rete rappresenta il messaggio più grande e ha una dimensione di **~1009 byte** (12B per il nonce GCM, 256B per la chiave simmetrica incapsulata, e ~741B per il payload JSON cifrato + tag 16B).

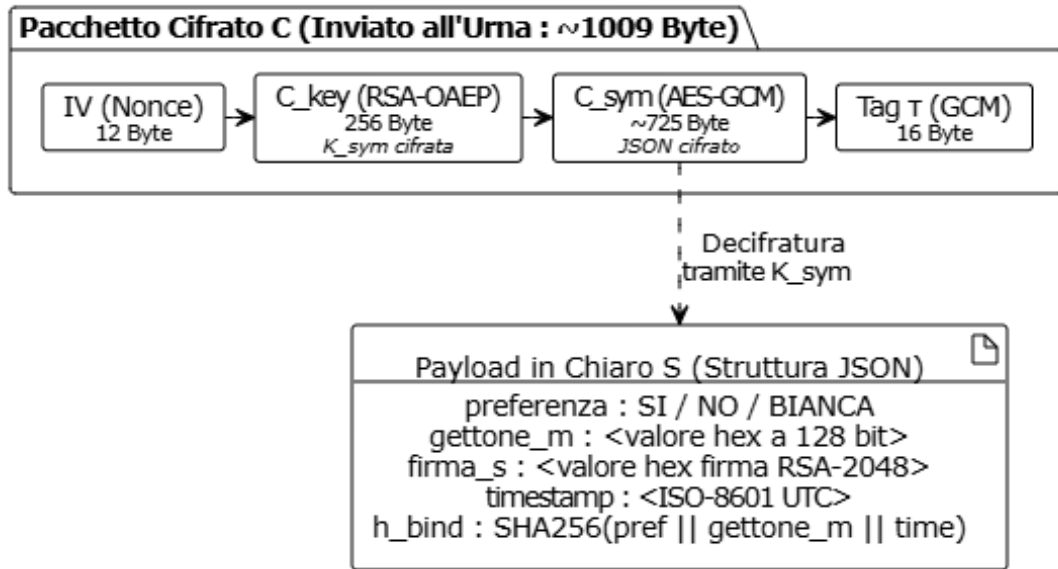


Figura 7: Struttura del pacchetto cifrato C generato dal Client Wallet: un'implementazione di uno schema ibrido AEAD dove C_{key} è inclusa come AAD nel calcolo del tag τ per vincolare crittograficamente la capsula al payload.

4.6 Prestazioni Sperimentali

I tempi sono stati misurati con `time.perf_counter()` (Python 3.12, cryptography backend OpenSSL) e rappresentano la mediana di più esecuzioni.

Operazione	Tempo Mediano (ms)
Generazione chiavi RSA Ateneo (2048 bit)	109.50
Generazione chiavi Urna + Shamir (3 quote)	147.50
Blinding lato client ($FDH + r^e \pmod n$)	39.99
Signing cieco server ($m'^d \pmod n$)	40.33
Unblinding lato client ($s' \cdot r^{-1} \pmod n$)	1.03
Cifratura ibrida (RSA-OAEP + AES-GCM)	0.91
Costruzione Merkle Tree (100 foglie)	0.01

Tabella 3: Costo Computazionale delle Singole Operazioni

Il costo crittografico complessivo end-to-end per il singolo utente si attesta a **~ 82 ms**, dominato dalle esponenziazioni modulari RSA.

4.6.1 Analisi di Scalabilità dello Scrutinio

Il **tempo totale** di scrutinio cresce linearmente con il numero di schede (da ~ 74 ms per 10 voti a ~ 1039 ms per 1000 voti), confermando la complessità $O(N)$. Tuttavia, il **costo marginale per singolo voto** decresce da 7.40 ms/voto a 1.04 ms/voto all'aumentare di N , poiché l'overhead fisso iniziale (ricostruzione Shamir, setup AES-GCM) viene ammortizzato su un batch più ampio. Con 1000 voti, l'operazione completa impiega ~ 1038 ms (circa **1 ms/voto**). Per un ateneo con 10.000 aventi diritto, il tempo stimato di scrutinio è nell'ordine dei 3–5 secondi.

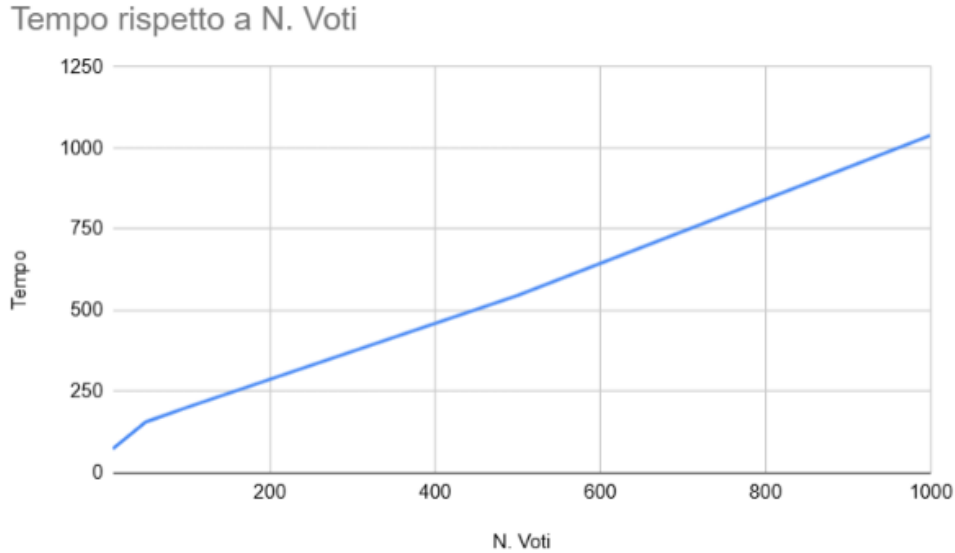


Figura 8: Scalabilità lineare delle prestazioni di scrutinio al variare del numero di schede. Il grafico mostra un incremento del tempo totale di elaborazione proporzionale al numero di voti, confermando la complessità computazionale asintotica $O(N)$ del protocollo implementato.

4.7 Validazione Funzionale e di Sicurezza

Il sistema è stato validato attraverso test unitari (`test_voting_system.py`):

- **Protezione Double Voting (TM.1):** Tentativi ripetuti di *blinding* vengono rifiutati dall’Ateneo (`test_double_voting_at_auth_server`).
- **Receipt-Freeness (TM.2/TM.4):** Invii multipli con lo stesso gettone m innescano la deduplicazione posticipata, mantenendo solo il voto con il `timestamp` più recente.
- **Resilienza Quorum Shamir:** Lo scrutinio è interrotto se vengono presentate meno di $t = 2$ quote valide (`test_quorum_not_reached`).
- **Integrità Payload:** La decifratura GCM scarta modifiche ai ciphertext, mentre il `h_bind` respinge alterazioni interne post-decifratura.

Nota sulla copertura — TM.5 (Crash Resilience): il WP3 identifica la resilienza a crash durante lo scrutinio (riesecuzione idempotente) come proprietà di sicurezza rilevante. Nell’implementazione attuale, lo scrutinio è un’operazione in-memory: un crash durante `tally()` richiede la rappresentazione delle quote Shamir e la rielaborazione del batch. Lo scrutinio è **idempotente di fatto**, poiché i pacchetti cifrati restano inalterati nell’array `encrypted_batch` e possono essere rielaborati producendo lo stesso risultato. Un test automatico che simuli esplicitamente un crash mid-tally con recovery non è incluso nella suite attuale, in quanto richiederebbe un meccanismo di persistenza su disco che esula dallo scope del prototipo in-memory.

4.8 Riepilogo

L’implementazione rispecchia fedelmente l’architettura crittografica del WP2. I test confermano che le proprietà teoriche di anonimato, unicità e anti-coercizione sono mantenute a livello applicativo. Le performance (pacchetti da $\sim 1\text{KB}$ e *tallying* da 1 ms per voto) risultano ottimali e perfettamente compatibili con scenari reali di consultazione universitaria.